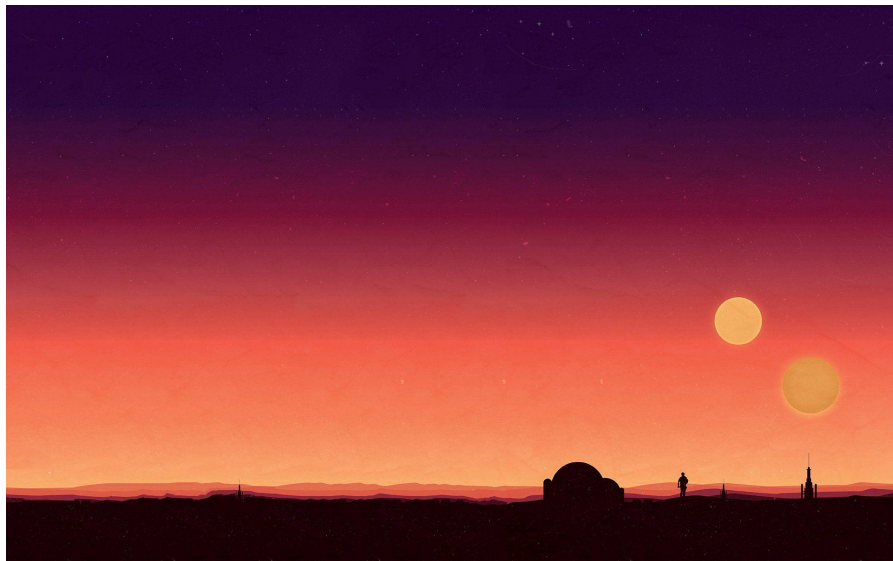


Senthil Sukumar

This is a Quarto website.

To learn more about Quarto websites visit <https://quarto.org/docs/websites>.



Assignment 2 - Load data into Spark, Create Tables and Queries

```
import os
import sys
from pathlib import Path
from pyspark.sql import SparkSession

os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
```

```

spark = (
    SparkSession.builder
    .appName("JobPostingsAnalysis")
    .master("local[*]")
    .config("spark.driver.host", "localhost")
    .config("spark.driver.memory", "4g")
    .config("spark.executor.memory", "4g")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate()
)
spark.conf.set("spark.sql.execution.arrow.pyspark.enabled", "false")

USE_SAMPLE = False
data_dir = Path("../data/MET_CareerCompass_2026")
if not data_dir.exists():
    data_dir = Path("data/MET_CareerCompass_2026")
parquet_files = sorted(data_dir.glob("*.parquet"))
if USE_SAMPLE:
    parquet_files = parquet_files[:5]

df = spark.read.parquet(*[str(path) for path in parquet_files])
df = df.toDF(*[column.lower() for column in df.columns])
df.cache()
df.createOrReplaceTempView("job_postings")

from pyspark.sql.functions import col, monotonically_increasing_id

# Industries dimension
industries_df = df.select(
    col("naics_2022_6"), col("naics_2022_6_name"),
    col("soc_5").alias("soc_code"), col("soc_5_name").alias("soc_name"),
    col("onet_name").alias("specialized_occupation"),
    col("soc_2_name").alias("occupation_group")
).distinct().withColumn("industry_id", monotonically_increasing_id())

industries_df = industries_df.select(
    "industry_id", "naics_2022_6", "naics_2022_6_name", "soc_code",
    "soc_name", "specialized_occupation", "occupation_group")

# Companies dimension
companies_df = df.select(
    col("company"), col("company_name"),

```

```

    col("company_raw"), col("company_is_staffing")
).distinct().withColumn("company_id", monotonically_increasing_id())

companies_df = companies_df.select(
    "company_id", "company", "company_name", "company_raw", "company_is_staffing")

# Locations dimension
locations_df = df.select(
    col("location"), col("city_name"), col("state_name"),
    col("county_name"), col("msa"), col("msa_name")
).distinct().withColumn("location_id", monotonically_increasing_id())

locations_df = locations_df.select(
    "location_id", "location", "city_name", "state_name", "county_name", "msa", "msa_name")

# Job Postings fact table
job_postings_df = (df
    .join(companies_df, on="company_name", how="left")
    .join(industries_df, on="naics_2022_6", how="left")
    .join(locations_df, on=["location", "city_name", "state_name"], how="left")
    .select(
        col("id"),
        col("title_clean"),
        col("company_id"),
        col("industry_id"),
        col("location_id"),
        col("employment_type_name"),
        col("remote_type_name"),
        col("body"),
        col("min_years_experience"), col("max_years_experience"),
        col("salary"), col("salary_from"), col("salary_to"),
        col("posted"), col("expired"), col("duration")
    )
    .dropDuplicates(["id"])
    .withColumn("job_postings_id", monotonically_increasing_id()))

job_postings_df = job_postings_df.select(
    "job_postings_id", "id", "title_clean", "company_id", "industry_id", "location_id",
    "employment_type_name", "remote_type_name", "body", "min_years_experience",
    "max_years_experience", "salary", "salary_from", "salary_to", "posted", "expired", "duration")

```

```
#industries_df.write.mode("overwrite").csv("./output/industries.csv", header=True)
#companies_df.write.mode("overwrite").csv("./output/companies.csv", header=True)
#locations_df.write.mode("overwrite").csv("./output/locations.csv", header=True)
#job_postings_df.write.mode("overwrite").csv("./output/job_postings.csv", header=True)

# Register SQL Tables
industries_df.createOrReplaceTempView("industries")
companies_df.createOrReplaceTempView("companies")
locations_df.createOrReplaceTempView("locations")
job_postings_df.createOrReplaceTempView("job_postings")
```

```
# Test Top 5 Job Titles query
top5_titles = spark.sql("""
    SELECT
        title_clean AS job_title,
        COUNT(*) AS posting_count
    FROM job_postings
    GROUP BY title_clean
    ORDER BY posting_count DESC
    LIMIT 5
""")

top5_pd = top5_titles.toPandas()
top5_pd.columns = ["Job Title", "Posting Count"]
top5_pd = top5_pd.reset_index(drop=True)
top5_pd.index += 1 # rank starts at 1
top5_pd
```

```
# Query 1: Industry-Specific Salary Trends Grouped by Job Title

spark.sql("""
    SELECT naics_2022_6, naics_2022_6_name, COUNT(*) as cnt
    FROM industries
    where naics_2022_6 = '518210'
    GROUP BY naics_2022_6, naics_2022_6_name
    ORDER BY cnt DESC
""").toPandas()

median_salary_df = spark.sql("""
    SELECT
        --i.naics_2022_6_name AS industry_name,
        i.specialized_occupation,
```

```

        PERCENTILE_APPROX(jp.salary_from, 0.5) AS median_salary
FROM job_postings as jp
JOIN industries as i ON jp.industry_id = i.industry_id
WHERE --i.naics_2022_6 in ('541900') -- can't find code '518210'
i.specialized_occupation in (
    'Business Intelligence Analysts',
    'Information Security Analysts',
    'SAP Analysts / Admin',
    'Data Scientists',
    'Market Research Analysts and Marketing Specialists',
    'Database Architects',
    'Data Warehousing Specialists',
    'Financial and Investment Analysts',
    'Data Quality Analysts')
    AND jp.salary_from IS NOT NULL
    AND jp.salary_from != ''
    AND TRY_CAST(jp.salary_from AS DOUBLE) > 0
GROUP BY --i.naics_2022_6_name,
i.specialized_occupation
ORDER BY median_salary DESC
""")

median_salary_pd = median_salary_df.toPandas()
print(f"Rows returned: {len(median_salary_pd)}")
median_salary_pd

import plotly.express as px

fig = px.bar(
    median_salary_pd,
    x="median_salary",
    y="specialized_occupation",
    color="specialized_occupation",
    orientation="h",
    title="Median Salary Trends in the Technology Sector by Specialized Occupation",
    labels={
        "median_salary": "Median Salary ($)",
        "specialized_occupation": "Specialized Occupation"
    },
    text="median_salary",
    color_discrete_sequence=px.colors.qualitative.Pastel
)

```

```

fig.update_traces(
    texttemplate="$%{text:,.0f}",
    textposition="inside"
)
fig.update_layout(
    showlegend=False,
    xaxis_tickformat="$,.0f",
    yaxis=dict(categoryorder="total ascending"),
    height=max(450, len(median_salary_pd) * 45),
    margin=dict(l=200, r=80, t=60, b=40),
    uniformtext_minsize=10,
    uniformtext_mode="hide"
)

fig.show()

```

```

# Query 2: Top Companies Hiring Remote Jobs in California
remote_ca_df = spark.sql(f"""
    SELECT
        c.company_name,
        COUNT(*) AS remote_jobs
    FROM job_postings jp
    JOIN companies c ON jp.company_id = c.company_id
    JOIN locations l ON jp.location_id = l.location_id
    WHERE jp.remote_type_name = 'Remote'
        AND l.state_name = 'California'
    GROUP BY c.company_name
    ORDER BY remote_jobs DESC
    LIMIT 5
    """)

remote_ca_pd = remote_ca_df.toPandas()
print(f"Rows returned: {len(remote_ca_pd)}")
remote_ca_pd

import seaborn as sns
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(13, 6))

sns.barplot(

```

```

    data=remote_ca_pd,
    x="company_name",
    y="remote_jobs",
    color="#5B7FA6",
    order=remote_ca_pd["company_name"],
    ax=ax
)

ax.set_title("Top 5 Companies Offering Remote Jobs in California",
             fontsize=13, pad=12)
ax.set_xlabel("Company Name", fontsize=11)
ax.set_ylabel("Number of Remote Jobs", fontsize=11)

# Rotate x-axis labels
ax.set_xticklabels(
    ax.get_xticklabels(),
    rotation=45,
    ha="right",
    fontsize=10
)

ax.yaxis.grid(True, linestyle="-", linewidth=0.5, color="#cccccc")
ax.set_axisbelow(True)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

fig.text(
    0.5, -0.04,
    "Solution 2, colors/details can change",
    ha="center", fontstyle="italic", fontsize=10, color="#555555"
)

plt.tight_layout()
plt.show()

```

```

# Query 3: Monthly Job Posting Trends in California
monthly_df = spark.sql("""
    SELECT
        YEAR(to_date(jp.posted, 'yyyy-MM-dd')) AS year,
        MONTH(to_date(jp.posted, 'yyyy-MM-dd')) AS month,
        COUNT(DISTINCT jp.id) AS job_count

```

```

FROM job_postings jp
JOIN locations l ON jp.location_id = l.location_id
WHERE l.state_name = 'California'
      AND to_date(jp.posted, 'yyyy-MM-dd') IS NOT NULL
GROUP BY year, month
ORDER BY year DESC, month DESC
""")

monthly_pd = monthly_df.toPandas()
print(f"Rows returned: {len(monthly_pd)}")
monthly_pd

import seaborn as sns
import matplotlib.pyplot as plt
import calendar

monthly_pd["year"] = monthly_pd["year"].astype(str)
monthly_pd = monthly_pd.sort_values(["year", "month"])

fig, ax = plt.subplots(figsize=(13, 6))

sns.lineplot(
    data=monthly_pd,
    x="month",
    y="job_count",
    hue="year",
    marker="o",
    linewidth=2,
    markersize=6,
    ax=ax
)

# Use only months present in data
actual_months = sorted(monthly_pd["month"].unique())
ax.set_xticks(actual_months)
ax.set_xticklabels([calendar.month_abbr[m] for m in actual_months], fontsize=10)

ax.set_title("Monthly Job Posting Trends in California", fontsize=13, pad=12)
ax.set_xlabel("Month", fontsize=11)
ax.set_ylabel("Number of Job Postings", fontsize=11)
ax.legend(title="Year", fontsize=10, title_fontsize=10)

```

```

ax.grid(True, linestyle="-", linewidth=0.5, color="#d0d0d0")
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()

```

```

# Query 4: Salary Comparisons Across Major US Cities
salary_metro_df = spark.sql("""
    SELECT
        CASE l.msa
            WHEN '47900' THEN 'Washington DC'
            WHEN '41860' THEN 'San Francisco'
            WHEN '42660' THEN 'Seattle'
            WHEN '26420' THEN 'Houston'
            WHEN '31080' THEN 'Los Angeles'
            WHEN '35620' THEN 'New York'
            WHEN '14460' THEN 'Boston'
            WHEN '34980' THEN 'Nashville'
            WHEN '12420' THEN 'Austin'
            WHEN '19100' THEN 'Dallas'
            WHEN '19740' THEN 'Denver'
            WHEN '28140' THEN 'Kansas City'
        END AS metro_area,
        ROUND(AVG(TRY_CAST(jp.salary_from AS DOUBLE)), 2) AS avg_salary,
        COUNT(*) AS job_count
    FROM job_postings jp
    JOIN locations l ON jp.location_id = l.location_id
    WHERE jp.salary_from IS NOT NULL
    AND jp.salary_from != ''
    AND TRY_CAST(jp.salary_from AS DOUBLE) > 0
    AND l.msa IN (
        '47900', '41860', '42660', '26420', '31080', '35620',
        '14460', '34980', '12420', '19100', '19740', '28140'
    )
    GROUP BY metro_area
    ORDER BY avg_salary DESC
""")

salary_metro_pd = salary_metro_df.toPandas()
print(f"Rows returned: {len(salary_metro_pd)}")
salary_metro_pd

```

```

import seaborn as sns
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(13, 8))

sns.barplot(
    data=salary_metro_pd,
    x="avg_salary",
    y="metro_area",
    color="#4B6C9E",                    # muted steel blue matching target
    order=salary_metro_pd["metro_area"], # preserve SQL sort order (desc)
    ax=ax
)

ax.set_title("Average Salary Comparisons Across Major US Metro Areas",
             fontsize=13, pad=12)
ax.set_xlabel("Average Salary ($)", fontsize=11)
ax.set_ylabel("Metro Area", fontsize=11)

ax.xaxis.grid(True, linestyle="--", linewidth=0.6, color="#cccccc")
ax.set_axisbelow(True)

for spine in ax.spines.values():
    spine.set_visible(True)

plt.tight_layout()
plt.show()

```